FVS Integration and Automation Framework

Project Requirements and Specifications

D&D Larix, LLC - Forest Analytics Division





WSU CPTS Capstone Team 14

Ziang Chen

Meshach Erisca

Tomas Stoker

9/30/2027

TABLE OF CONTENTS

I. Introduction 3

II. Team Members – Bios and Project Roles 3

III. System Requirements Specification 3

III.1. Use Cases 3

III.2. Functional Requirements 4

III.2.1. FR-01 Analytical Lineage 4

III.2.2. FR-02 Input Validation 5

III.2.3. FR-03 Output Validation 5

III.2.4. FR-04 Software Boundary 5

III.2.5. FR-05 Temporary memory 5

III.2.6. FR-06 Baseline Run 5

III.2.7. FR-07 Multilevel Error Handling 6

III.2.8. FR-08 Alternative Scenarios 6

III.3. Non-Functional Requirements 6

III.4. User Stories and Traceability Matrix 6

III.4.1. User Stories 6

III.4.2. Traceability matrix 8

III.5. Standards 9

IV. System Evolution 9

V. Glossary 10

VI. References 10

I. Introduction
FRASS combines forest inventory and site evidence with biological and economic analysis to support forest management decisions. The Forest Vegetation Simulator (FVS) supplies growth projections FRASS needs, but the two systems require a reliable way to exchange data. This project will validate an approved forest stand, run FVS using R and rFVS, and return projected tree and stand data to FRASS for subsequent analysis [1].

II. Team Members – Bios and Project Roles
The team has assigned a team lead but has not yet divided the technical components. All members are currently contributing to requirements and project planning. Specific implementation responsibilities will be assigned after further meetings and once a more comprehensive development plan is created.

Ziang Chen — Team Lead. Ziang is a Computer Science major with an Economics minor at Washington State University. His experience includes systems programming, simulations, Git, Docker, and CI/CD. As team lead, Ziang coordinates client communication, meetings, and team planning while also contributing to the project's technical work.

Meshach Erisca — Team Member. Meshach is a Computer Science major with experience in C++, C#, Python, JavaScript, and Windows and Linux development. Projects including a PostScript interpreter and a team application prototype have provided experience in software implementation and collaboration. Meshach will contribute to development and testing; specific component ownership is still being determined.

Tomas Stoker — Team Member. Tomas studies Computer Science and has experience with C/C++, C#, multithreading, and database-backed applications. Previous projects involving mapping and data management are relevant to the FRASS–FVS integration. Tomas will contribute to development and testing; specific component ownership is still being determined.

III. System Requirements Specification
In this section you will describe the features, functions and other specifications that are requirements for your product. You will also specify the client/stakeholder need(s) that requirement maps to. If you find a stakeholder that your first assignment did not identify, feel free to add them here as required.

Please refer to Section 4.4 in the CptS 322 &&/|| CptS 422 textbooks like "Object-Oriented Software Engineering" to refresh your knowledge on software requirements.

For the system requirements of the integration layer, the system needs to prepare and validate stand evidence, execute FVS runs, and return projected data to FRASS. The returned data should retain the information to reproduce each result, and we should be able to trace where data came from. The use case below shows the main workflow and the following sections specify functional and non functional requirements.

III.1. Use Cases
Table 01: UC – 01: Initiate Base Run

Actors:	User

Pre-condition:	Stand data within FRASS database
Post Condition:	Simulation Report is generated based on user parameters
Path:	1. Initiate Run 2. Initialize Input Table 3. Build Input from Internal Database 4. Validate Input 5. Start External Run 6. Validate Output 7. Save Results Handoff Control to Main System

A diagram is depicted in Figure 1 to illustrate the use case.

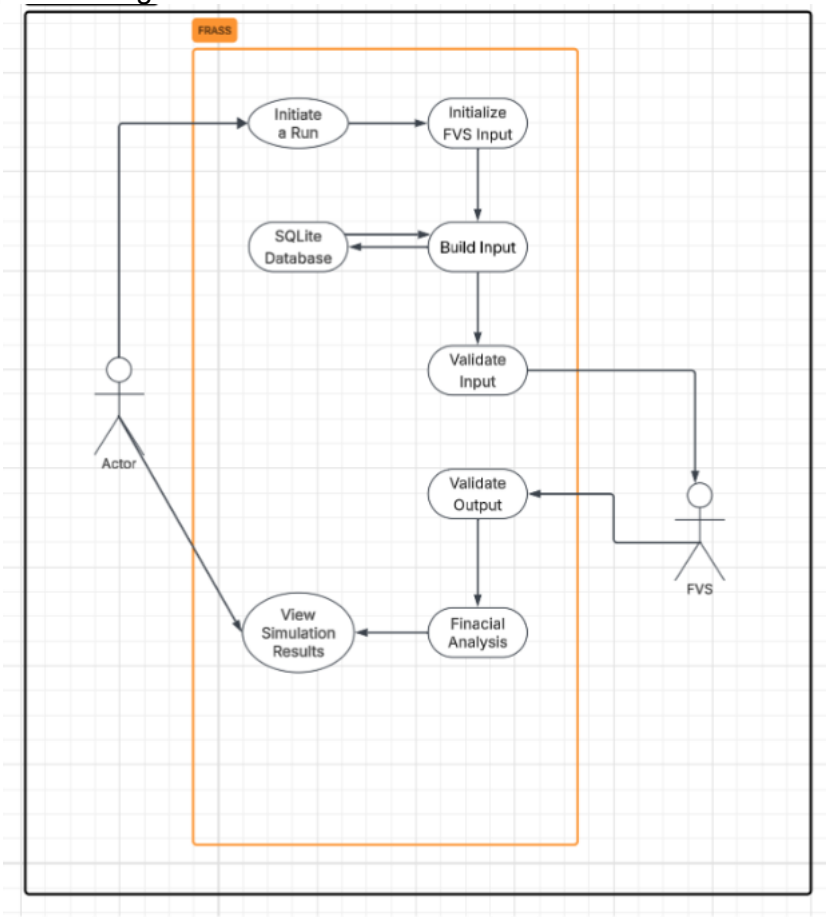


Figure 1: Use case diagram

III.2. Functional Requirements

III.2.1. FR-01 Analytical Lineage

Description:	The system must be able to draft, store, and produce findings, including Source ID and GIS site data regardless of the Origin or Variant used.
Source:	Client project specification [1, 2]

Priority:	Level 0

III.2.2. FR-02 Input Validation

Description	The system must prepare the input and check that fields required by FVS are either filled or explicitly flagged.
Source:	Client project specification [2]
Priority:	Level 0

III.2.3. FR-03 Output Validation

Description:	The system must not assume that the simulation data is correct and validate it for anomalies.
Source:	Client project specification [1,2]
Priority:	Level 0

III.2.4. FR-04 Software Boundary

Description:	The system must keep a clear division of FRASS and FVS and not impose feature reimplementation from past or future expansion while maintaining this division.
Source:	Client project specification [1]
Priority:	Level 1

III.2.5. FR-05 Temporary memory

Description:	Candidate states with initial baseline or alternative parameters must live in memory, and not create permanent database entries
Source:	Client project specification [1]
Priority:	Level 1

III.2.6. FR-06 Baseline Run

Description:	System must support “Year 0” simulation for input agreement with downstream systems

Source:	Client project specification [1]
Priority:	Level 1

III.2.7. FR-07 Multilevel Error Handling

Description:	System must handle exceptions and discard invalid states at any point in the pipeline
Source:	Client project specification [1]
Priority:	Level 2

III.2.8. FR-08 Alternative Scenarios

Description:	System must be able to support alternatives to administration and management changes with the same initial baseline, then allow the user to select the desired run.
Source:	Client project specification [2]
Priority:	Level 2

III.3. Non-Functional Requirements

Non-Functional Requirement	Description
NFR-01 Reliability	The system shall prevent failed or invalid FVS runs from being treated as valid results.
NFR-02 Performance	The system shall support evaluation of candidate projections without requiring every candidate to be permanently stored.
NFR-03 Usability	The system shall communicate portfolio readiness and run errors in terms FRASS users can understand.
NFR-04 Maintainability	The integration's data contract, configuration, and version dependencies shall be documented for future developers.

III.4. User Stories and Traceability Matrix

III.4.1. User Stories

US-01: Initiate a Base Run

As a FRASS User
I want to run FVS using an approved stand
So that I can obtain projected tree and stand conditions.

Feature: Base projection

Scenario: Ready portfolio

- Given: The stand has approved inventory, site evidence, and projection settings
- When: I initiate the run
- Then: FRASS shall execute FVS and validate the returned results
- And: The projected conditions shall be available for downstream FRASS analysis

Scenario: Incomplete portfolio

- Given: Required evidence is missing or conflicting
- When: I request the run
- Then: FRASS shall identify the problem
- And: The run shall not proceed until the problem is resolved

US-02: Run an Alternative Scenario

As a FRASS user
I want to change permitted projection or management settings
So that I can examine a different future from the same approved starting condition.

Feature: Alternative projections

Scenario: Valid alternative

- Given: An approved starting condition and valid alternative settings
- When: I initiate another run
- Then: FRASS shall execute the alternative
- And: Its result shall remain distinguishable from earlier runs

US-03: Review a Projection

As a FRASS user
I want to see the evidence and settings associated with a result
So that I can explain or reproduce the projection.

Feature: Result traceability

Scenario: Review a valid result

- Given: A validated projection is available
- When: I open its details

- Then: FRASS shall show its stand, source inventory, site evidence, settings, and software versions
- And: I shall be able to identify the starting condition used for the run

III.4.2. Traceability matrix

The table below maps functional requirements to respective use cases and user stories.

Functional Requirement	Use Case	User Stories	Priority
FR-01: Analytical Lineage	UC-01: Connect the completed run to its input and results.	US-03: Review a projection.	Level 0
FR-02: Input Validation	UC-01: Validate the portfolio before execution.	US-01: Initiate a base run, including the incomplete-portfolio scenario.	Level 0
FR-03: Output Validation	UC-01: Validate FVS results before returning them to FRASS.	US-01: Initiate a base run.	Level 0
FR-04: Software Boundary	UC-01: Invoke FVS through R/rFVS and return biological results.	US-01: Initiate a base run.	Level 1
FR-05: Temporary Memory	Indirectly supports UC-01 when candidate results are processed; it is not a separate user action.	US-02: Run an alternative scenario.	Level 1
FR-06: Baseline Run	UC-01: Establish the approved Year Zero starting condition.	US-01: Initiate a base run; US-02: Run an alternative scenario.	Level 1
FR-07: Multilevel Error Handling	UC-01: Follow the exception path when validation or execution fails.	US-01: Incomplete-portfolio scenario.	Level 2
FR-08: Alternative Scenarios	Not covered by UC-01; a separate use case is optional.	US-02: Run an alternative scenario.	Level 2

III.5. Standards

The integration will follow the documented FRASS–FVS data contract and the applicable FVS/rFVS input and output conventions. The team will define units, identifiers, version recording, and validation rules explicitly. No additional regulatory or industry standard is specified and any required standard will be added here once confirmed [1, 2].

IV. System Evolution

This specification assumes that the GIS-Lite and inventory teams will provide identifiable, approved stand evidence and that a supported rFVS/FVS interface will be available on the FRASS server. Changes to upstream data fields, FVS variants or versions, server infrastructure, or downstream FRASS needs may require updates to the data contract and validation rules. Keeping those rules and version information explicit will make future changes easier to detect and manage [1].

V. Glossary

Term	Definition
FRASS	Forest Resource Analysis System Software; the system that manages forest evidence and uses projections in later analyses.
FVS	Forest Vegetation Simulator; a model that projects how trees and forest stands develop over time.
FVS Portfolio	The approved stand inventory, site evidence, variant, and settings assembled for an FVS run.
Stand	An area of forest treated as one unit for inventory and biological modeling.
Inventory	Recorded observations describing the current forest, including stand and tree measurements.
Site evidence	Location and environmental information used to describe the stand's growing conditions.
FVS variant	A regional version of FVS appropriate to a particular geographic area.
Year Zero	The approved current forest condition used as the starting point for a projection.
R	The programming environment used to control the interaction with FVS.
rFVS	The R interface used to communicate with the FVS modeling engine.
DLL/API	The compiled FVS software library and the defined operations used to call it.
Analytical lineage	The connection between a result and the source evidence, settings, and software versions that produced it.
GIS-Lite	The FRASS component that supplies spatial and site information for a stand.
Mobile Flex Cruiser	A field application that collects forest inventory observations.

VI. References

[1] W. E. Schlosser, "FRASS FVS Integration Capstone Charter: Direct FVS Execution Through R, rFVS, and the FVS DLL/API," D&D Larix, LLC, Sept. 2, 2026.

[2] W. E. Schlosser, *Engineering FRASS: Scientific Workflow and System Architecture, Part II: From Forest Evidence to Defensible Findings*, D&D Larix, LLC, Sept. 2026.